

Express Framework

B. Vijay Kumar

Express Framework : Content

- Introduction to Express
- Installation of Express
- Create first Express application
- request, and response objects
- configuring an Express application
- Rendering views
- sessions
- forms
- file upload
- Connecting to SQL Database

Introduction

- Express.js is a Node.js web framework that is quick, adaptable, and simple.
- It is essentially a tool that makes it easier to create server-side JavaScript web apps and APIs.

Why learn Express?

- Express is a user-friendly framework that simplifies the development process of Node applications.
- It uses JavaScript as a programming language and provides an efficient way to build web applications and APIs.
- With Express, you can easily handle routes, requests, and responses, which makes the process of creating robust and scalable applications much easier.

Introduction

Features

- **Minimalist Framework:** Express provides a thin layer of fundamental web application features, making it easy to set up a server without a lot of overhead.
- **Middleware Support:** Middleware functions in Express allow you to process requests and responses, handle errors, and add additional functionality. You can use built-in middleware or create your own.
- **Routing:** Express offers a powerful routing system that allows you to define routes for different HTTP methods (GET, POST, PUT, DELETE) and URL patterns.
- **Templating Engines:** You can integrate various templating engines (like Pug, EJS, or Handlebars) to render dynamic HTML views.
- **RESTful API Creation:** Express is ideal for building RESTful APIs, providing the necessary tools to manage routes and handle JSON data.
- **Community and Ecosystem:** It has a large community and a vast ecosystem of middleware and plugins that extend its functionality.

Installation

Installing Express involves just a few steps.

Prerequisites:

- Node.js: Ensure you have Node.js installed.
- npm: Node Package Manager (npm) is installed automatically with Node.js. Check if it's installed

Steps to Install Express:

- Create a New Directory for Your Project: Open your terminal:

```
mkdir firstapp  
cd firstapp
```

- Initialize a New Node.js Project: Run the following command to create a package.json file:

```
npm init -y
```

Installation

- Install Express: Use npm to install Express:

```
npm install express
```

- Verify Installation: After installation, you can check your package.json file or the `node_modules` directory to ensure Express is installed.

```
npm list express
```

First Application

Once you have done the above task do the following:

- **Create Your First Express Server:** create a new file app.js and write the following code:

```
const express = require('express');
const app = express();
const PORT = 3000;
app.get('/', (req, res) => {
    res.send('Hello, World!');
});
app.listen(PORT, () => {
    console.log(`Server running at http://localhost:${PORT}`);
});
```

- **Run Your Application:**

```
node app.js
```

Request and Response Objects

Request Object (req): The request object contains information about the incoming HTTP request. It includes various properties and methods that you can use to interact with the request. Check below:

- **req.params:** An object containing route parameters. For example, in the route `/users/:id`, `req.params.id` will contain the value of `id`.
- **req.query:** An object containing the URL query string parameters. For example, in the URL `/search?term=node`, `req.query.term` will contain the value `node`.
- **req.body:** An object containing the parsed request body (for POST requests). To access this, you usually need to use middleware like `express.json()` or `express.urlencoded()`.
- **req.method:** The HTTP method used for the request (GET, POST, etc.).
- **req.url:** The full URL of the request.
- **req.headers:** An object containing the request headers.

Request and Response Objects

Example:

```
const express = require('express');
const app = express();
const PORT = 3000;

app.get('/students/:id', (req, res) => {
  const userId = req.params.id;
  res.send('User ID: ${userId}');
});

app.get('/search', (req, res) => {
  const searchTerm = req.query.term;
  res.send('Search Term: ${searchTerm}');
});

app.listen(PORT, () => {
  console.log('Server is running on http://localhost:${PORT}');
});
```

Request and Response Objects

Open the URL using postman software to test it as below:

The screenshot displays the Postman application interface. On the left sidebar, the 'My Workspace' section is active, showing a 'Your collection' dialog box with an 'Authorization' dropdown set to 'API Key'. Below this, a message states: 'Create a collection for your requests. A collection lets you group related requests and easily set common authorization, tests, scripts, and variables for all requests in it.' A 'Create Collection' button is visible.

The main workspace shows a GET request to `http://localhost:3000/students/123`. The request is configured with the following tabs: Params, Auth, Headers (6), Body, Scripts, Tests, and Settings. The 'Query Params' section is currently selected, showing a table with two columns: 'Key' and 'Value', and a 'Description' column. The table is empty.

The response section shows a status of '200 OK' with a response time of '3 ms' and a size of '239 B'. The response body is displayed in 'Pretty' format, showing a single line: `1 User ID: 123`.

Request and Response Objects

Response Object (res): The response object is used to send a response back to the client. It includes various properties and methods that you can use to interact with the request. Check below:

- `res.send(data)`: Sends the HTTP response with the specified data, automatically setting the appropriate content type.
- `res.json(data)`: Sends a JSON response, setting the content type to `application/json`.
- `res.status(code)`: Sets the HTTP status code for the response.
- `res.redirect(url)`: Redirects the client to a different URL.
- `res.render(view, locals)`: Renders a view template (for server-side rendering).

Request and Response Objects

Example:

```
const express = require('express');
const app = express();
app.use(express.json()); // Middleware to parse JSON bodies

app.get('/', (req, res) => {
  res.send('Hello, World!');
});

app.get('/user/:id', (req, res) => {
  const userId = req.params.id;
  // Simulate fetching user data
  const user = { id: userId, name: 'Vijay' };
  res.json(user); // Send JSON response
});

app.post('/user', (req, res) => {
  const newUser = req.body; // Get user data from the
  request body
  // Simulate saving user data
  res.status(201).json(newUser); // Send back created user
```

Request and Response Objects

Example:(Continued....)

```
});  
app.listen(3000, () => {  
  console.log('Server running on http://localhost:3000');  
});
```

Configuration

Configuration: Configuring an Express application involves setting up the necessary components, middleware, and routes for your application to function properly.

Here are the Steps involved in configuring a basic Express application:

- **Initialize a new Node.js project:** Open your terminal and run the following command to create a new directory and initialize a Node.js project.

```
mkdir myfirstexpressapp  
cd myfirstexpressapp  
npm init -y
```

- Install Express and any other necessary packages using npm.

```
npm install express
```

Configuration

- **Set Up Routes:** In `routes/index.js`, define a simple route.

```
// routes/index.js
const express = require('express');
const router = express.Router();

// Define a GET route
router.get('/', (req, res) => {
  res.send('Hello, World!');
});

// Export the router
module.exports = router;
```

Configuration

- **Create Your Express Application:** In app.js, set up the main Express application.

```
// app.js
const express = require('express');
const path = require('path');
const logger = require('morgan'); // Optional logging
    middleware
const indexRouter = require('./routes/index'); // Ensure
    this line is correct

const app = express();
const PORT = process.env.PORT || 3000;

// Middleware
app.use(logger('dev')); // Logs requests to the console
app.use(express.json()); // Parses JSON request bodies
// Parses URL-encoded request bodies
app.use(express.urlencoded({ extended: false }));
```

Configuration

```
// Serve static files from the 'public' directory
app.use(express.static(path.join(__dirname, 'public')));

// Route setup
app.use('/', indexRouter); // Ensure this is correct

// Error handling middleware (optional)
app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(500).send('Something broke!');
});

// Start the server
app.listen(PORT, () => {
  console.log(`Server is running on http://localhost:${PORT}`);
});
```

Configuration

Run your application: In your terminal, execute the following command to start your Express server:

```
node app.js
```

Rendering Views

Render Views in Express application: To render views in express application we need to do the following:

- Install a View Engine(EJS): EJS stands for Embedded JavaScript, is a simple templating language that allows developers to generate HTML markup with plain JavaScript.

```
npm install ejs
```

- Create a directory named views in the root of your project.
- Inside the views directory, create an index.ejs file: and write the following:

```
<!-- views/index.ejs -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width,
initial-scale=1.0">
  <title><%= title %></title>
```

Rendering Views

```
</head>
<body>
<h1><%= message %></h1>
</body>
</html>
```

- **Set Up the View Engine in Express:** Create a new file app.js in root directory with the following code:

```
// app.js
const express = require('express');
const app = express();
const port = 3000;

// Set the view engine to EJS
app.set('view engine', 'ejs');

// Set the directory for views (default is 'views')
app.set('views', './views');
```

Rendering Views

```
// Routes
app.get('/', (req, res) => {
  res.render('index', { title: 'Home', message: 'Hello,
    World!' });
});

app.listen(port, () => {
  console.log('Server running at http://localhost:${port}');
});
```

- Start the Server and Test:

```
node app.js
```

Sessions

Sessions in Express: Sessions in Express allow you to store user data across requests, which is useful for managing user authentication and other stateful interactions.

- Install express-session:

```
npm install express-session
```

- **Configure Sessions in Express:** Create a new file app.js in root directory and do the following:

```
// app.js
const express = require('express');
const session = require('express-session');
const app = express();
const port = 3000;
```

Sessions

```
// Configure session middleware
app.use(
  session({
    secret: 'your-secret-key', // Replace with a secure,
    random secret
    resave: false,             // Don't save session if
    unmodified
    saveUninitialized: false,  // Don't create session until
    something is stored
    cookie: {
      maxAge: 60000             // Set cookie expiration time
      in ms (optional)
    }
  })
);

app.get('/', (req, res) => {
  res.send('Session example');
});
```

Sessions

```
app.listen(port, () => {  
  console.log('Server running at http://localhost:${port}');  
});
```

Storing and Accessing Session Data:

- Storing Data in the Session:

```
app.get('/login', (req, res) => {  
  // Simulating login by storing user info in the  
  session  
  req.session.user = { username: 'Vijay', role: 'admin'  
  };  
  res.send('User logged in!');  
});
```

Sessions

- Accessing Session Data

```
app.get('/dashboard', (req, res) => {  
  if (req.session.user) {  
    res.send('Welcome, ${req.session.user.username}!');  
  } else {  
    res.send('Please log in first.');
```

- Destroying Session Data

```
app.get('/logout', (req, res) => {  
  req.session.destroy((err) => {  
    if (err) {  
      return res.send('Error logging out');    }  
    res.send('You are logged out!');  });  
});
```

Forms

In **Express.js**, forms are a key part of building web applications as they allow users to send data to the server.

Steps how to implement it:

- Create an sample app handling forms
- npm install express
- create a file app.js
- write the following code:

```
const express = require('express');
const app = express();
const port = 3000;

app.use(express.urlencoded({ extended: true }));
// Parse URL-encoded data
app.use(express.json()); // Parse JSON data
```

Forms

```
app.get('/', (req, res) => {  
    res.send('<form action="/submit" method="POST"><  
input name="data"><button>Send</button></form>');  
});  
  
app.post('/submit', (req, res) => {  
    res.send('Received: ${req.body.data}');  
});  
  
app.listen(port, () => console.log('App running at  
http://localhost:${port}'));
```

- Now run the code node app.js

Forms

Output Screens:

- Default Screen:



A screenshot of a web form. It consists of a single-line text input field containing the text "vijay". To the right of the input field is a button labeled "Send".

- After Submit:

Received: vijay

Fileupload

Handling file uploads in **Express.js** is straightforward with the help of middleware like multer.

Steps how to implement it:

- Create an sample app handling forms
- npm install express
- npm install multer : Multer is a middleware for handling multipart/form-data, primarily used for file uploads in express.
- create a file app.js
- write the following code:

```
const express = require('express');  
const multer = require('multer');  
const path = require('path');  
  
const app = express();  
const port = 3000;
```

Fileupload

```
// Set storage options for Multer
const storage = multer.diskStorage({
  destination: (req, file, cb) => {
    cb(null, 'uploads/'); // Directory where files will
    be saved
  },
  filename: (req, file, cb) => {
    cb(null, Date.now() + path.extname(file.originalname)); // File name
  },
});

// Initialize Multer with storage options
const upload = multer({
  storage: storage,
  limits: { fileSize: 5 * 1024 * 1024 }, // Limit file
  size to 5MB
});
```

Fileupload

```
// Serve a simple HTML form
app.get('/', (req, res) => {
  res.send(`
    <form action="/upload" method="POST" enctype="
multipart/form-data">
      <input type="file" name="file" />
      <button type="submit">Upload</button>
    </form>
  `);
});

// Route for file upload
app.post('/upload', upload.single('file'), (req, res) => {
  if (!req.file) {
    return res.status(400).send('No file uploaded.');

}



res.send('File uploaded successfully: ${req.file.  
filename}');



});


```

Fileupload

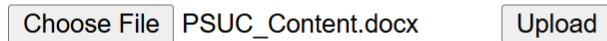
```
// Start the server
app.listen(port, () => {
  console.log('Server running at http://localhost:${port}');
});
```

- Now run the code node app.js

Forms

Output Screens:

- Default Screen:



A screenshot of a web form's default state. It features a light gray button labeled "Choose File" followed by the text "PSUC_Content.docx" and another light gray button labeled "Upload".

- After Submit:

File uploaded successfully: 1732060900050.docx

Questions

◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡ 🔍 ↺